

Disciplina:

Desenvolvimento para Dispositivos Móveis

Objetivos:

Compreender a linguagem de programação.

Programar aplicativos.

Testar aplicativos.

Documentar.

Conteúdo

- Histórico.
- Arquitetura da plataforma.
- Estrutura do ambiente de desenvolvimento.
- Comandos da plataforma.
- Desenvolvimento de aplicativos.
- Integração com sistemas legados.
- Interação com servidores.

Histórico da plataforma

- Novo SO: Fuchsia (2016)
- Framework: Flutter
- Linguagem de programação: Dart
- Novas plataformas: Android, iOS, Windows, macOS, Linux (2018)
- Web: 2021

O que significa: Build for any screen?

- Acesse: <https://flutter.dev>

Single codebase

- Mobile
- Web
- Desktop
- Embedded (Toyota, LG webOS)

Fast

- Compilado

Productive

- Atualiza o código e vê as mudanças.
- Mantém o estado atual.

Flexible

- Controle tudo.

Powered by Dart

- A linguagem usada é Dart: acesse <https://dart.dev>

Dart

Introdução: <https://dart.dev/language>

Crie os programas em <https://dartpad.dev>

Hello World

Mostre:

```
Hello, Barretos!
```

Hello World

Mude o nome de **main** para **inicio**, funciona?

Variables

Use `print` para imprimir:

- `name`
- `year`
- `antennaDiameter`
- `flybyObjects`
- `flybyObjects.first`
- `image`
- `image['url']`

Control flow statements

Usando as variáveis anteriores:

- O que o `if` imprime?
- O que o primeiro `for` imprime?
- O que o segundo `for` imprime?
- O que o `while` imprime?

Functions

Como criar uma função que recebe um número e multiplica o número por 2, complete no código:

```
// TODO 1: seu código aqui!  
  
void main() {  
    var numero = 3;  
    var resultado = porDois(numero);  
    print(resultado);  
}
```


Functions

Como fica a função `porDois()` usando **arrow** (`=>`)?

Comments

Adicione comentários explicando o propósito de cada função:

```
int divTres(var x) {  
    return x / 3;  
}  
  
void main() {  
    print(divTres(9));  
}
```

Imports

Para usar a constante `pi` da biblioteca `math`, use `import` `'dart:math'`:

```
double area(var raio) {  
    return pi * raio * raio;  
}  
  
void main() {  
    print(area(5));  
}
```

Imports

Use `pow()` da biblioteca `dart:math` ao invés de `raio * raio`.

Classes

```
class Aluno {  
    String nome;  
    double nota1, nota2, nota3;  
  
    // Construtor  
    Aluno(this.nome, this.nota1, this.nota2, this.nota3);  
  
    double get notaFinal => (nota1 + nota2 + nota3) / 3;  
  
    void situacao() {  
        print('Aluno: $nome');  
    }  
}  
  
void main() {  
    var primeiro = Aluno('Fulano', 0, 0, 0);  
    primeiro.situacao();  
}
```

função `main`

Todo programa Dart começa pela função `main()`.

`void` é o tipo de retorno quando a função não retorna nada.

`()` a lista de argumentos está vazia.

Digite tudo em <https://dartpad.dev>:

```
void main() {  
    print('Hello, Dart!');  
}
```

```
// sem início: main não encontrado!  
void inicio() {  
    print('Primeiro código');  
}
```

Exemplo

```
void segunda() {  
    print('Segunda');  
    terceira();  
}  
  
void terceira() {  
    print('Terceira');  
}  
  
void main() {  
    print('Primeira');  
    segunda();  
}
```



```
print()
```

Imprime um objeto no console (tela preta).

Exemplo

```
void main() {  
    print('Primeira linha');  
    print('Segunda linha');  
    print('Terceira linha');  
}
```

Comentários

Partes comentadas não executam, são ignoradas.

```
// Comentários para explicar o código  
/* explicando  
em várias  
linhas */
```

Variáveis

É necessário especificar: tipo, nome e valor de uma variável.

```
tipo nome = valor; // Criando nome na memória.
```

Exemplo

```
void main() {  
    String nome = 'Fulano';  
    print(nome);  
  
    nome = 'Fulano de Tal'; // pode ser alterada.  
    print(nome);  
}
```

String

Texto na memória possui o tipo String.

```
void main() {  
    String nome = 'Fulano de Tal'; // Pode usar " ou '  
    print(nome);  
}
```

Unindo Strings

Para juntar Strings use `+`:

```
void main() {  
    String nome = 'Fulano'  
    String sobrenome = "de Tal";  
    String completo = nome + sobrenome;  
  
    print(completo);  
}
```

Exemplo

Crie um programa em Dart que declara duas variáveis `cidade` com o valor `"Barretos"` e `estado` com o valor `'São Paulo'`. Mostre a mensagem abaixo usando dois comandos `print()` e as variáveis criadas:

```
Cidade: Barretos  
Estado: São Paulo
```


Inteiros

O tipo `int` armazena números inteiros.

```
void main() {  
    int idade = 25;  
    print(idade);  
}
```

int

Inteiros permitem cálculos:

```
void main() {  
    int nota = 3;  
    print(nota);  
  
    nota = nota + 5; // 3 + 5  
    print(nota);  
  
    int bonus = 2;  
  
    int total = nota + bonus; // 8 + 2  
    print(total);  
}
```

Exemplo

Altere o código anterior para mostrar mensagens na apresentação da nota:

```
Nota: 3  
Nota aumentada: 8  
Nota total: 10
```

Valores embutidos em Strings

Podemos adicionar os valores das variáveis em Strings usando a sintaxe: `"Valor: $nomeDaVariavel"`

```
void main() {  
    int idade = 25;  
    print("Idade: $idade");  
}
```

tipo **double**

Números com pontos decimais.

```
void main() {  
    double altura = 1.71;  
    print(altura);  
}
```

Cálculos com double

```
void main() {  
    int lados = 3;  
    int metade = lados ~/ 2; //  
    print(metade);  
  
    double vertices = 3.0;  
    double meio = vertices / 2;  
    print(meio);  
}
```

class num

Veja os [operadores](<https://api.flutter.dev/flutter/dart-core/num-class.html#operators>) disponíveis com num.

bool

Valores do tipo `bool`: `true` ou `false`.

```
void main() {  
    bool chove = true;  
    print(chove);  
}
```



```
void main() {  
    bool tarefaCompleta = false;  
    print("Terminou a tarefa: $tarefaCompleta");  
  
    tarefaCompleta = true;  
    print("Estado atual da tarefa: $tarefaCompleta");  
}
```

var

Podemos criar variáveis com `var` e deixar o Dart descobrir o tipo.

```
void main() {  
    var nome = 'Fulano';  
    print(nome);  
}
```

Outros tipos

```
void main() {  
    var preco = 1.99;  
    print(preco * 10);  
  
    var quantidade = 9;  
    print(quantidade - 1);  
  
    var disponivel = false;  
    print(!disponivel);  
}
```

final

Uma variável `final` não pode ser alterada após a inicialização.

```
void main() {  
    final nome = 'Fulano'; // nome é string  
    print(nome);  
}
```

final não aceita novo valor

```
void main() {  
    final quantidade = 9;  
    // quantidade = 8;  
    print(quantidade);  
}
```

final e tipos

```
void main() {  
    final int base = 8;  
    print(base);  
}
```

const

Uma variável `const` não pode mudar. E deve ter o valor definido no código.

O valor deve estar definido na hora da compilação.

```
void main() {  
    const nome = 'Fulano'; // nome é string  
    print(nome);  
}
```

tipo com const

Dart descobre o tipo da variável quando `const` é usada na criação.

```
void main() {  
  const nota = 7.2; // double  
  print(nota*1.2);  
}
```


Convenções de nomes de variáveis

- Use *camel/Case* para variáveis.
- Comece com letras ou sublinhados.
- Use nomes significativos.
- Letras minúsculas para constantes.

Null Safety

As variáveis devem conter um valor antes de serem usadas:

```
void main() {  
    String nome;  
    // Error: Non-nullable variable 'nome' must be assigned before it can be used.  
    print(nome);  
}
```

marca para null

Para permitir o uso de valor `null`, o tipo deve conter `?`:

```
void main() {  
    String? nome;  
    print(nome);  
}
```

Valor null

Qualquer tipo pode ser null:

```
void main() {  
    int? idade = 22;  
    print(idade);  
  
    idade = null;  
    print(idade);  
}
```

Operadores aritméticos

- 
- 
- 
- 

Exercício

- Crie um novo programa Dart.
- Crie a variável `nome` com o valor `'Camiseta ADS'` dentro de `main()`.
- Crie a variável `quantidade` com o valor `3`.
- Crie a variável `preco` com o valor `49.90`.
- Crie a variável `disponivel` com o valor `true`.
- Crie a constante `imposto` com o valor `5.0`.

- Calcule o preço total usando a fórmula: `(quantidade * preco) * (1 + imposto/100)`
- Mostre as mensagens usando `print()` e as variáveis:

```
Produto: Camiseta ADS  
Disponível em estoque: true  
Quantidade: 3  
Preço unitário: 49.90  
Valor total: R$ NN,NN
```

Envio

<https://forms.gle/pPKeFp5Zphh9Upec9>